

High-Performance Order Execution and Management System

1.0

Generated by Doxygen 1.9.1

1 Class Index	1
1.1 Class List	1
2 File Index	3
2.1 File List	3
3 Class Documentation	5
3.1 DeribitClient Class Reference	5
3.1.1 Detailed Description	6
3.1.2 Constructor & Destructor Documentation	6
3.1.2.1 DeribitClient()	6
3.1.3 Member Function Documentation	7
3.1.3.1 cancel_all() [1/2]	7
3.1.3.2 cancel_all() [2/2]	7
3.1.3.3 cancel_order()	7
3.1.3.4 close()	8
3.1.3.5 connect()	8
3.1.3.6 generate_id()	8
3.1.3.7 get_orderbook()	8
3.1.3.8 get_positions()	9
3.1.3.9 handle_error()	9
3.1.3.10 init_logger()	9
3.1.3.11 list_subscriptions()	10
3.1.3.12 modify_order()	10
3.1.3.13 place_order()	10
3.1.3.14 send_auth()	11
3.1.3.15 subscribe_book()	11
3.1.3.16 unsubscribe()	12
3.2 MessageParser Class Reference	12
3.2.1 Detailed Description	12
3.2.2 Constructor & Destructor Documentation	13
3.2.2.1 MessageParser()	13
3.2.3 Member Function Documentation	13
3.2.3.1 parse_and_print()	13
4 File Documentation	15
4.1 src/deribit_client.cpp File Reference	15
4.2 src/message_parser.cpp File Reference	15
4.2.1 Detailed Description	15

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

DeribitClient	Client class for interacting with the Deribit cryptocurrency exchange API	5
MessageParser	Parser for handling Deribit WebSocket API messages Provides methods to parse and handle different types of responses from the exchange	12

Chapter 2

File Index

2.1 File List

Here is a list of all documented files with brief descriptions:

include/ deribit_client.hpp	??
include/ message_parser.hpp	??
include/ prompts.hpp	??
include/ websocket_server.hpp	??
src/ deribit_client.cpp	
Implementation of the DeribitClient class for interacting with Deribit exchange	15
src/ message_parser.cpp	
Implementation of the MessageParser class for handling Deribit API messages	15

Chapter 3

Class Documentation

3.1 DeribitClient Class Reference

Client class for interacting with the Deribit cryptocurrency exchange API.

```
#include <deribit_client.hpp>
```

Public Member Functions

- [DeribitClient](#) (const std::string &host, const std::string &port, const std::string &target, uint16_t server_port)
Constructs a new [DeribitClient](#).
- void [connect](#) ()
Establishes a WebSocket connection to the Deribit server.
- void [start_receiving](#) ()
Starts the message receiving thread.
- void [start_sending](#) ()
Starts the message sending thread.
- void [send_auth](#) (const std::string &api_key, const std::string &api_secret)
Authenticates with the Deribit API.
- void [subscribe_book](#) (const std::string &instrument)
Subscribes to orderbook updates for an instrument.
- void [cancel_order](#) (const std::string &order_id)
Cancels an order by ID.
- void [close](#) ()
Closes the WebSocket connection.
- void [cancel_all](#) (const std::string &instrument_type)
Cancels all orders for a specific instrument type.
- void [get_orderbook](#) (const std::string &instrument_name, int depth=10)
Gets the orderbook for an instrument.
- void [get_positions](#) (const std::string ¤cy, const std::string &kind="any")
Gets positions for a currency.
- void [unsubscribe](#) (const std::string &symbol)
Unsubscribes from a symbol's updates.
- std::string [generate_id](#) (const std::string &function_name)
Generates a unique message ID.

- void `list_subscriptions` () const
Lists all active subscriptions.
- void `process_messages` ()
Message processing thread that handles received messages.
- void `send_messages` ()
Message sending thread that processes outgoing messages.
- void `start_message_processing` ()
- void `handle_error` (const std::string &context, const std::exception &e)
Handles and logs errors that occur during operations.
- void `place_order` (const std::string &instrument, std::optional< double > amount=std::nullopt, std::optional< double > contracts=std::nullopt, std::string type="limit", std::optional< double > price=std::nullopt, std::optional< std::string > time_in_force=std::nullopt, std::optional< bool > post_only=std::nullopt, std::optional< bool > reduce_only=std::nullopt, std::optional< std::string > label=std::nullopt, std::optional< double > trigger_price=std::nullopt, std::optional< std::string > trigger=std::nullopt)
Places an order on the exchange.
- void `modify_order` (const std::string &order_id, std::optional< double > amount=std::nullopt, std::optional< double > price=std::nullopt, std::optional< std::string > post_only=std::nullopt, std::optional< std::string > time_in_force=std::nullopt, std::optional< std::string > label=std::nullopt)
Modifies an existing order.
- void `cancel_all` (std::optional< std::string > type=std::nullopt, std::optional< std::string > instrument_name=std::nullopt, std::optional< std::string > kind=std::nullopt)
Cancels all orders matching criteria.
- void `start_server` ()
Starts the local WebSocket server.
- void `stop_server` ()
Stops the local WebSocket server.

Static Public Member Functions

- static void `init_logger` ()
Initializes the logging system.

Public Attributes

- bool `is_authenticated` = false

3.1.1 Detailed Description

Client class for interacting with the Deribit cryptocurrency exchange API.

3.1.2 Constructor & Destructor Documentation

3.1.2.1 DeribitClient()

```
DeribitClient::DeribitClient (
    const std::string & host,
    const std::string & port,
    const std::string & target,
    uint16_t server_port )
```

Constructs a new `DeribitClient`.

Constructs a new `DeribitClient` with SSL context and logging setup.

Parameters

<i>host</i>	The hostname of the Deribit server
<i>port</i>	The port number for the connection
<i>target</i>	The WebSocket endpoint path
<i>server_port</i>	The local server port number

3.1.3 Member Function Documentation

3.1.3.1 `cancel_all()` [1/2]

```
void DeribitClient::cancel_all (
    const std::string & instrument_type )
```

Cancels all orders for a specific instrument type.

Parameters

<i>instrument_type</i>	The type of instrument
------------------------	------------------------

3.1.3.2 `cancel_all()` [2/2]

```
void DeribitClient::cancel_all (
    std::optional< std::string > type = std::nullopt,
    std::optional< std::string > instrument_name = std::nullopt,
    std::optional< std::string > kind = std::nullopt )
```

Cancels all orders matching criteria.

Parameters

<i>type</i>	Optional order type filter
<i>instrument_name</i>	Optional instrument filter
<i>kind</i>	Optional kind filter

3.1.3.3 `cancel_order()`

```
void DeribitClient::cancel_order (
    const std::string & order_id )
```

Cancels an order by ID.

Parameters

<i>order_</i> <i>_id</i>	The order ID to cancel
-----------------------------	------------------------

3.1.3.4 close()

```
void DeribitClient::close ( )
```

Closes the WebSocket connection.

Gracefully closes the WebSocket connection and joins threads.

3.1.3.5 connect()

```
void DeribitClient::connect ( )
```

Establishes a WebSocket connection to the Deribit server.

Establishes secure WebSocket connection and starts message handling threads.

3.1.3.6 generate_id()

```
std::string DeribitClient::generate_id (
    const std::string & function_name )
```

Generates a unique message ID.

Parameters

<i>function_name</i>	The name of the calling function
----------------------	----------------------------------

Returns

A unique message identifier

3.1.3.7 get_orderbook()

```
void DeribitClient::get_orderbook (
    const std::string & instrument_name,
    int depth = 10 )
```

Gets the orderbook for an instrument.

Parameters

<i>instrument_name</i>	The instrument name
<i>depth</i>	The depth of the orderbook (default: 10)

3.1.3.8 get_positions()

```
void DeribitClient::get_positions (
    const std::string & currency,
    const std::string & kind = "any" )
```

Gets positions for a currency.

Parameters

<i>currency</i>	The currency to get positions for
<i>kind</i>	The kind of positions (default: "any")

3.1.3.9 handle_error()

```
void DeribitClient::handle_error (
    const std::string & context,
    const std::exception & e )
```

Handles and logs errors that occur during operations.

Parameters

<i>context</i>	Description of where the error occurred
<i>e</i>	The exception that was caught

3.1.3.10 init_logger()

```
void DeribitClient::init_logger ( ) [static]
```

Initializes the logging system.

Initializes and configures the logging system with file and console outputs.

3.1.3.11 list_subscriptions()

```
void DeribitClient::list_subscriptions ( ) const
```

Lists all active subscriptions.

Lists all current subscriptions to the console.

3.1.3.12 modify_order()

```
void DeribitClient::modify_order (
    const std::string & order_id,
    std::optional< double > amount = std::nullopt,
    std::optional< double > price = std::nullopt,
    std::optional< std::string > post_only = std::nullopt,
    std::optional< std::string > time_in_force = std::nullopt,
    std::optional< std::string > label = std::nullopt )
```

Modifies an existing order.

Parameters

<i>order_id</i>	The order ID to modify
<i>amount</i>	Optional new amount
<i>price</i>	Optional new price
<i>post_only</i>	Optional post-only setting
<i>time_in_force</i>	Optional new TIF
<i>label</i>	Optional new label

3.1.3.13 place_order()

```
void DeribitClient::place_order (
    const std::string & instrument,
    std::optional< double > amount = std::nullopt,
    std::optional< double > contracts = std::nullopt,
    std::string type = "limit",
    std::optional< double > price = std::nullopt,
    std::optional< std::string > time_in_force = std::nullopt,
    std::optional< bool > post_only = std::nullopt,
    std::optional< bool > reduce_only = std::nullopt,
    std::optional< std::string > label = std::nullopt,
    std::optional< double > trigger_price = std::nullopt,
    std::optional< std::string > trigger = std::nullopt )
```

Places an order on the exchange.

Places an order with specified parameters.

Parameters

<i>instrument</i>	The instrument to trade
<i>amount</i>	Optional amount in currency
<i>contracts</i>	Optional number of contracts
<i>type</i>	Order type (e.g., "limit", "market")
<i>price</i>	Optional limit price
<i>time_in_force</i>	Optional TIF instruction
<i>post_only</i>	Optional post-only flag
<i>reduce_only</i>	Optional reduce-only flag
<i>label</i>	Optional order label
<i>trigger_price</i>	Optional trigger price
<i>trigger</i>	Optional trigger type

Note

All optional parameters are handled through `std::optional`

3.1.3.14 send_auth()

```
void DeribitClient::send_auth (
    const std::string & api_key,
    const std::string & api_secret )
```

Authenticates with the Deribit API.

Sends authentication request to the Deribit API.

Parameters

<i>api_key</i>	The API key from Deribit
<i>api_secret</i>	The API secret from Deribit
<i>api_key</i>	Client API key
<i>api_secret</i>	Client API secret

3.1.3.15 subscribe_book()

```
void DeribitClient::subscribe_book (
    const std::string & instrument )
```

Subscribes to orderbook updates for an instrument.

Subscribes to order book updates for a specific instrument.

Parameters

<i>instrument</i>	The instrument identifier (e.g., "BTC-PERPETUAL")
<i>instrument</i>	The instrument to subscribe to (e.g., "BTC-PERPETUAL")

3.1.3.16 unsubscribe()

```
void DeribitClient::unsubscribe (
    const std::string & symbol )
```

Unsubscribes from a symbol's updates.

Unsubscribes from order book updates.

Parameters

<i>symbol</i>	The symbol to unsubscribe from
---------------	--------------------------------

The documentation for this class was generated from the following files:

- include/deribit_client.hpp
- src/deribit_client.cpp

3.2 MessageParser Class Reference

Parser for handling Deribit WebSocket API messages Provides methods to parse and handle different types of responses from the exchange.

```
#include <message_parser.hpp>
```

Public Member Functions

- [MessageParser](#) ()
Constructs a [MessageParser](#) and initializes message handlers.
- void [parse_and_print](#) (const std::string &raw_msg)
Parses a raw message and routes it to appropriate handler.

3.2.1 Detailed Description

Parser for handling Deribit WebSocket API messages Provides methods to parse and handle different types of responses from the exchange.

3.2.2 Constructor & Destructor Documentation

3.2.2.1 MessageParser()

```
MessageParser::MessageParser ( )
```

Constructs a [MessageParser](#) and initializes message handlers.

Constructs [MessageParser](#) and initializes handler function map.

3.2.3 Member Function Documentation

3.2.3.1 parse_and_print()

```
void MessageParser::parse_and_print (
    const std::string & raw_msg )
```

Parses a raw message and routes it to appropriate handler.

Parses JSON message and routes to appropriate handler.

Parameters

<i>raw_msg</i>	The raw JSON message string from WebSocket
----------------	--

Extracts function name from message ID and calls corresponding handler

Parameters

<i>raw_msg</i>	Raw JSON string from WebSocket
----------------	--------------------------------

The documentation for this class was generated from the following files:

- include/message_parser.hpp
- src/[message_parser.cpp](#)

Chapter 4

File Documentation

4.1 src/deribit_client.cpp File Reference

Implementation of the [DeribitClient](#) class for interacting with Deribit exchange.

```
#include "deribit_client.hpp"
#include <boost/asio/connect.hpp>
#include <iostream>
Include dependency graph for deribit_client.cpp:
```

4.2 src/message_parser.cpp File Reference

Implementation of the [MessageParser](#) class for handling Deribit API messages.

```
#include "message_parser.hpp"
#include <iostream>
#include <iomanip>
Include dependency graph for message_parser.cpp:
```

Typedefs

- using **json** = nlohmann::json

4.2.1 Detailed Description

Implementation of the [MessageParser](#) class for handling Deribit API messages.

